

Vibe Coding: A Practical Field Guide

Vibe Coding is not just “asking AI to write code.” It is a way to collaborate with AI agents as working members of a software team. This guide covers the full loop: starting a project, giving agents context, reviewing their output, managing long sessions, using subagents, and shipping safely.

Table of Contents

1. What Is Vibe Coding?
 2. Specs: The Starting Point
 3. What Goes Into AGENTS.md or CLAUDE.md
 4. Cold Starts: Joining or Creating a Project
 5. Context Management
 6. Using Subagents
 7. Agent Workflows and Collaboration Patterns
 8. .gitignore: The Hygiene File
 9. Git Worktrees and Codex Worktrees
 10. Creating Skills
 11. System Prompts vs User Prompts
 12. CI/CD for Agent-Written Code
 13. Testing Code and Testing Agent Behavior
 14. Advanced Principles
 15. A Complete Workflow Example
 16. Anti-Patterns Checklist
-

1. What Is Vibe Coding?

The phrase “vibe coding” became popular after Andrej Karpathy described a style of programming where you rely heavily on AI, accept most generated code, and paste errors back when something breaks. In real engineering work, the idea becomes more serious and more disciplined.

The practical version is this:

You are not only writing code. You are directing one or more AI agents that write code.

Your main work shifts from typing every character to managing four things.

1.1 Describe Intent Clearly

Do not say:

Build login.

Say:

Add email + OTP login using the existing auth module.

Store OTP codes in Redis, expire them after 5 minutes, and reuse the current session creation logic.

Good intent tells the agent:

- what to build
- where it belongs
- which existing system it must respect
- how success will be verified

1.2 Provide the Right Context

Agents do not automatically know your project conventions, hidden constraints, deployment history, or past mistakes. You need to give that context in durable forms:

- AGENTS.md or CLAUDE.md
- feature specs
- architecture docs
- file paths instead of pasted files
- examples of good local code
- test data or ground truth cases

The quality of the context usually matters more than the model choice.

1.3 Review, Steer, and Correct

This is most of the work.

Review means you inspect what the agent changed:

- read the diff
- run tests or start the app
- check logs and browser output
- ask why a design was chosen
- challenge new dependencies and abstractions

Never accept “it should work” as proof. Make the agent show command output, or run the command yourself.

Steer means you set boundaries before the agent starts:

Before editing files, tell me:

- your implementation plan
- which files you expect to touch
- what decisions you need from me
- what risks or edge cases you see

This catches bad direction early.

Useful constraints:

Do not introduce new dependencies.
Do not change public method signatures.
Do not edit migrations.
Keep the existing error handling style.

Correct means you interrupt when the direction is wrong:

Stop. This is going in the wrong direction. Reset the plan and explain a smaller approach.

Letting a wrong session continue usually creates more code to revert and more context pollution.

1.4 Decompose, Compress, and Switch Agents at the Right Time

Vibe coding requires knowing when to stop:

- split large work into phases
- open a new session after a milestone
- write a handoff file before context gets too full
- use a subagent for broad search or independent research
- commit frequently so git becomes your undo system

1.5 From Managing Characters to Managing Attention

Traditional programming is character-level control. You type variables, brackets, function calls, and tests.

With agents, the characters are generated by the agent. Your job is to manage the agent's attention:

- What should it focus on now?
- Which files should it read?
- Which constraints must stay active?
- Which old topic should be cleared?
- When should the session be reset?

The agent's context window is a limited budget. If you fill it with old logs, irrelevant files, and stale decisions, it will perform worse. Vibe coding is largely the practice of spending that attention budget well.

2. Specs: The Starting Point

A spec is a clear description of what you want the agent to build. It can be a short chat message, a Markdown document, or an issue.

2.1 What Makes a Good Spec

A good spec has three properties.

Clear intent

```
Add email + OTP login to the existing auth module.
```

is better than:

```
Build login.
```

Explicit constraints

Mention style, dependencies, performance, security, compatibility, error handling, and areas that must not be touched.

Testable acceptance criteria

The agent needs to know what “done” means:

- which endpoint returns what
- which command must pass

- which UI flow must work
- which edge cases must be covered

2.2 Lightweight Specs

Use lightweight specs for small changes:

```
Add UserService.deleteAccount:
- soft delete by setting deleted_at
- revoke all sessions for that user
- return NotFound if the user does not exist
- add unit tests for success and missing user
```

This is enough when the task is small and local.

2.3 Heavyweight Specs

Use a document for work that spans multiple sessions, days, or agents:

Account Deletion

Background

The product supports disabling accounts but not true deletion. We need a compliant deletion.

Goals

- Users can request deletion.
- A 30-day grace period allows cancellation.
- After 30 days, data is deleted or anonymized.

Non-Goals

- Ownership transfer for published content is not part of this phase.

Technical Plan

To be drafted and reviewed before implementation.

Acceptance Criteria

- [] POST /account/deletion creates a deletion request.
- [] Cancellation works during the grace period.
- [] Background cleanup is idempotent.
- [] Tests cover success, cancellation, missing user, and retry behavior.

The spec is not a one-time artifact. Update it when you discover a constraint, make a decision, or defer a scope item.

3. What Goes Into AGENTS.md or CLAUDE.md

AGENTS.md and CLAUDE.md are project-level operating manuals for coding agents. Put them at the repository root unless a subdirectory needs more specific rules.

3.1 Project Identity

Project Overview

This is a B2B SaaS backend.
Stack: Go 1.22, PostgreSQL, Redis, AWS ECS Fargate.
The frontend lives in a separate repository.

The agent should quickly understand what kind of system it is in.

3.2 Directory Map

This is usually the most valuable section:

Code Structure

- ``cmd/api/`` - HTTP endpoint
- ``internal/auth/`` - authentication; read SECURITY.md before changing
- ``internal/billing/`` - billing; use decimal for money, never float
- ``pkg/`` - public reusable packages; change carefully
- ``migrations/`` - database migrations; only add new files, never edit old ones

Agents make better changes when they know ownership boundaries.

3.3 Coding Conventions

Coding Rules

- Wrap errors with ``fmt.Errorf("doing X: %w", err)``.
- Use ``slog`` for logging, not ``fmt.Println``.
- Use table-driven tests.
- Do not add comments unless they explain a non-obvious decision.

Write rules that are specific enough to change behavior.

3.4 Commands

Commands

- Test: ``make test``
- Start local dev: ``make dev``
- Run migrations: ``make migrate-up``
- Lint: ``make lint``

Do not make the agent guess command names.

3.5 Red Lines

Do Not

- Do not edit existing migration files.
- Do not commit without running tests.
- Do not add TODOs in production code.
- PII fields must go through ``internal/crypto/pii.go``.

Red lines should be direct and unambiguous.

3.6 Lessons Learned

This section captures project-specific traps:

Known Pitfalls

- PostgreSQL stores timestamps in UTC, but API responses use the user's timezone. Use ``utils/tz.go``; do not manually call ``.In()`` everywhere.
- Redis keys must use the namespace helper in ``internal/cache/keys.go``.

Every time an agent repeats a mistake, add the lesson here.

3.7 What Not to Put There

Avoid:

- long architecture essays
- one-off feature requirements
- personal preferences unrelated to the project
- giant pasted files

Keep AGENTS.md short enough to be useful. Around 200-400 lines is often enough for a real project.

4. Cold Starts: Joining or Creating a Project

Cold starts are where agents can help the most, but also where they can do the most damage if they start coding too early.

4.1 Brand-New Projects

Start with a spec:

I want to build X. Do not write code yet.

Help me draft a spec and ask every important question before choosing a technical design.

Then ask for options:

Based on the spec, propose three technical approaches.

Compare tradeoffs. Do not create files yet.

After the choice is clear:

- initialize the project skeleton
- create AGENTS.md
- write setup commands
- commit small milestones

The first AGENTS.md does not need to be perfect. It needs to exist and improve as the project teaches you things.

4.2 Existing Projects

When joining someone else's project, do not start with:

Add feature X.

Start with discovery:

Scan the repository and draft AGENTS.md.

Do not modify application code.

Identify framework, commands, directory responsibilities, and risky areas.

Then go deeper:

Act like a code archaeologist. Do not edit files.

Answer:

1. What are the main modules and dependencies?
2. Which files look most complex?
3. What does recent git history suggest is fragile?
4. Where is test coverage weak?
5. What implicit conventions can you infer?

Next, run the app locally:

Help me get this project running locally.

When an error appears, report it exactly. Do not guess configuration.

Record every manual step so we can update AGENTS.md.

Finally, do a low-risk warmup task:

- fix a typo
- add a log line
- add a small test
- update documentation

This gives both you and the agent a safe full loop: read, edit, test, commit.

5. Context Management

Agent context is limited. Long sessions degrade.

Common symptoms:

- the agent repeats mistakes you already corrected
- it forgets important constraints
- it invents files that do not exist

- tool use becomes slower or less precise
- output becomes vague

5.1 When to Switch or Compress

Switch at natural stopping points:

- a feature phase is complete
- a bug is fixed and the next task is unrelated
- backend work is done and you are moving to UI
- exploration is done and implementation is next

Do not wait until the agent becomes bad. Switch before the session gets heavy.

Rough context usage guide:

- 0-40%: normal work
- 40-65%: be aware
- 65-80%: prepare a handoff
- above 80%: write a handoff immediately and start fresh

5.2 Built-In Session Commands

Different tools use different names, but the patterns are similar:

Command	Purpose	Use When
<code>/compact</code>	summarize and keep going	you need more room in the same session
<code>/clear</code>	clear current conversation	you want a clean topic in the same window
<code>/init</code>	scan project and draft agent instructions	first step in an existing repo
<code>/resume</code>	continue a previous session	unfinished work continues later

`/compact` is useful, but it lets the AI decide which details survive. A handoff file is often safer because you choose what gets preserved.

5.3 Emergency Recovery

If the session is already overloaded, stop adding work. Ask for a handoff:

Pause. Before we lose context, write docs/notes/handoff.md with:

- what we are doing
- the spec or issue link
- files changed
- tests run and results
- current blockers
- next recommended step

Then start a new session:

Read AGENTS.md, the spec, and docs/notes/handoff.md.
Continue from the previous session, but restate the plan before editing.

5.4 Prevention

Good habits:

- one session per clear task or phase
- commit after each meaningful milestone
- store decisions in files, not only chat
- reference file paths instead of pasting large content
- send broad searches to subagents
- close solved topics explicitly

Context is not long-term memory. Files are.

6. Using Subagents

A subagent is a separate agent with its own context, assigned a specific task. It returns a result to the main agent without polluting the main context with every file it read.

6.1 Why Use Subagents

Suppose the main agent is implementing payments, but it needs to find every use of an old Payment-Client.

If the main agent reads 50 files, its context gets filled with search noise.

If a subagent does it, the main agent receives:

Found 12 usages:

- path A: old constructor
- path B: direct method call
- path C: test fixture

The main context stays focused.

6.2 Good Subagent Tasks

Use subagents for:

- large codebase searches
- isolated research
- independent test writing
- parallel module changes
- verification that can run while you implement
- gathering evidence from logs or CI output

6.3 Bad Subagent Tasks

Avoid subagents for:

- tiny tasks
- tasks that require the main conversation context
- work that needs many rounds of user clarification
- tightly coupled edits to the same files

6.4 Write Subagent Instructions Like Function Signatures

Bad:

Look around and tell me what you find.

Good:

Task: Search `src/` for direct reads of `process.env`.

Input: repository files only.

Output: list lines as `path:line - variable name`.

Constraints: do not edit files; ignore node_modules.

Completion: include total count and any risky usage patterns.

Subagents work best when the task is bounded and the output format is fixed.

7. Agent Workflows and Collaboration Patterns

Many “agent systems” fail because they confuse two ideas:

- workflow: the steps are predefined by you
- agent: the model decides the next step dynamically

Use workflows whenever the steps are predictable. Use autonomous agents only when the path is genuinely unknown.

7.1 Workflow vs Agent

Question	Workflow	Agent
Who decides the steps?	you or code	the LLM
Structure	fixed steps, maybe branches	dynamic loop
Predictability	high	lower
Debuggability	easier	harder
Cost	controlled	can expand
Best for	known processes	open-ended tasks

Decision test:

Can I know the steps in advance?

If yes, use a workflow. If not, use an agent with guardrails.

7.2 Five Core Workflow Patterns

Pattern 1: Prompt Chaining Output from one step becomes input to the next:

```
draft spec -> review spec -> design -> review design -> implement -> test
```

Use this when each step can be checked before moving on.

Example:

```
Step 1: Read specs/001.md and extract a checklist.  
Step 2: Wait for my confirmation.  
Step 3: Design the implementation for each checklist item.  
Step 4: Wait for my confirmation.  
Step 5: Implement one item at a time.
```

Pattern 2: Routing Classify the input, then send it to a specialized path:

```
PR files -> language router  
          -> Go reviewer  
          -> TypeScript reviewer  
          -> Python reviewer
```

Use routing when one prompt for all cases would become too vague.

Pattern 3: Parallelization Split independent work and run it at the same time:

```
module A tests  
module B tests  
module C tests  
-> aggregate results
```

Use this only when tasks do not edit the same files or depend on each other.

Voting is another form:

```
Run the same behavior test 3 times.  
If one run fails, mark the prompt unstable.
```

Pattern 4: Orchestrator-Workers The orchestrator decides the subtasks after seeing the input:

```
request -> orchestrator reads repo -> creates worker tasks -> aggregates results
```

This is the pattern behind many subagent systems.

Use it when you cannot know the exact subtasks before inspection, but once discovered, each subtask is independent.

Pattern 5: Evaluator-Optimizer One agent generates, another evaluates, and feedback loops until a threshold is met:

```
generator -> candidate  
candidate -> evaluator  
if fail -> feedback to generator  
if pass -> final output
```

Set an exit condition:

- maximum 3 iterations
- score $\geq 9/10$
- evaluator must explicitly output PASS

Without an exit condition, this pattern can loop forever.

7.3 When to Use a True Agent

Use an autonomous agent when:

- the number of steps is unknown
- the next action depends on tool output
- the task requires reading, testing, editing, and retesting
- “done” is based on judgment rather than a fixed transform

Examples:

- debug a production-like failure
- review a complex PR
- implement a feature in an unfamiliar codebase

Guardrails:

- restrict tools to what is needed
- set max iterations or budget
- log every action
- require confirmation for destructive actions
- fail clearly when ambiguous

7.4 Multi-Agent Collaboration Modes

Specialist agents

Different agents or models handle different strengths:

- implementation
- deep reasoning
- documentation lookup
- security review
- test writing

Role-based agents

Same model, different instructions:

- architect
- coder
- reviewer
- tester
- documentation writer

This helps because a coder tends to justify its own output, while a reviewer is instructed to find problems.

Adversarial review

Ask one agent to attack another agent’s design:

You are reviewing this proposal. Do not praise it.
List concrete risks, missing cases, and simpler alternatives.

Hierarchical collaboration

A manager agent delegates to leads or workers. Use sparingly. More than two layers usually multiplies error and coordination cost.

7.5 Use Files as the Protocol Layer

Agents should exchange structured files instead of chatting endlessly:

```
.
├── AGENTS.md
├── specs/
│   ├── 001-auth.md
│   └── 002-billing.md
├── docs/
│   ├── architecture.md
│   ├── decisions/
│   └── notes/
├── .agent/
│   ├── tasks/
│   ├── reviews/
│   ├── handoffs/
│   └── runs/
└── src/
```

Files give you:

- persistence across sessions
- auditability
- clearer handoffs
- fewer forgotten decisions

7.6 Decision Framework

Can one LLM call solve it?

yes -> use one call

no -> continue

Can the steps be known in advance?

yes -> workflow

fixed sequence -> prompt chaining

classification -> routing

independent parts -> parallelization

dynamic split -> orchestrator-workers

quality iteration -> evaluator-optimizer

no -> agent with guardrails

Do you need multiple agents?

different skills -> specialist
different responsibilities -> role-based
critical review -> adversarial
large hierarchy -> avoid unless necessary

7.7 Workflow Anti-Patterns

Anti-Pattern	Result	Fix
using an agent when a workflow is enough	expensive and hard to debug	define steps
asking an agent to review itself	self-justifying review	use another role
agent-to-agent free chat	context bloat	exchange files
deep agent hierarchy	compounding errors	keep it flat
no exit condition	infinite loops	max iterations
parallel tasks with dependencies	conflicts	sequence them

8. .gitignore: The Hygiene File

.gitignore tells git which files should not be tracked.

Example:

```
# dependencies
node_modules/
.venv/

# build output
dist/
build/
*.pyc
__pycache__

# environment
.env
.env.local

# editor and OS files
.DS_Store
.idea/
.vscode/

# logs and cache
*.log
.cache/
```

8.1 Why It Matters

Your repository should contain source files, not machine-specific artifacts.

Do not commit:

- dependency directories
- build output
- local environment files
- logs
- caches
- secrets
- editor settings that are personal

Without a good `.gitignore`, agents may stage garbage or secrets when they run `git add`.

8.2 Patterns

```
node_modules/      # directory
*.log              # every .log file
/config.json       # root config.json only
!important.log     # exception
**/temp/           # temp directory at any depth
```

8.3 Common Trap: Already Tracked Files

Adding a file to `.gitignore` does not untrack it if it is already committed.

```
git rm --cached .env
git commit -m "remove env file from tracking"
```

If a secret was committed, rotate the secret immediately. Removing it from the latest commit is not enough if it remains in history.

8.4 Agent-Specific Ignores

For agent-heavy projects:

```
.agent/runs/
.agent/cache/
runs/
*.handoff.md.tmp
.llm-cache/
```

Do not ignore durable project knowledge:

```
AGENTS.md
CLAUDE.md
specs/
docs/
skills/
```

`.gitignore` is project hygiene. It keeps `git status` readable and reduces the chance of unsafe commits.

9. Git Worktrees and Codex Worktrees

Multiple agents editing the same working directory will eventually collide:

- shared git index
- branch switching conflicts
- overwritten files
- dev server port conflicts
- dependency or lockfile churn

Git worktrees solve the filesystem part.

9.1 What Is a Git Worktree?

A worktree is another checkout of the same repository with its own working directory and branch.

```
my-repo/  
  src/  
  
../wt/auth/  
  src/      # branch agent/auth  
  
../wt/billing/  
  src/      # branch agent/billing
```

Each agent works in its own directory.

9.2 Basic Commands

```
mkdir -p ../wt  
  
git worktree add -b agent/auth ../wt/auth main  
git worktree add -b agent/billing ../wt/billing main  
  
git worktree list  
  
git worktree remove ../wt/auth  
git branch -D agent/auth
```

9.3 The Biggest Worktree Trap

Ignored files do not appear in a new worktree:

- node_modules/
- .env
- .venv/
- build output
- caches

So a new worktree may not run immediately.

Create a setup script:

```
#!/usr/bin/env bash
set -euo pipefail

cp ../../main-repo/.env .
npm ci
```

Document it:

Worktree Setup

In a new worktree, run ``bash scripts/setup-worktree.sh`` before testing.
Each worktree must use a different dev server port.

9.4 When to Use Worktrees

Use worktrees when:

- tasks are independent
- you want A/B implementation experiments
- multiple agents need to work at the same time
- reviewer and coder agents should run separately

Do not use worktrees when:

- every task edits the same files
- setup cost is too high
- you only have one small task

9.5 Parallel Worktree Workflow

1. Split the work into independent specs.
2. Check file overlap. If overlap exists, sequence the work.
3. Create one worktree per spec.
4. Run setup in each worktree.
5. Assign one agent per worktree.
6. Review each diff.
7. Run tests.
8. Merge or rebase carefully.
9. Remove worktrees.

The bottleneck is still human review bandwidth. More agents are not useful if nobody can review the output.

10. Creating Skills

A skill is a reusable, parameterized workflow template for an agent.

Create a skill when you have explained the same task pattern at least three times.

Examples:

- add a REST API endpoint
- create a database migration
- write a weekly engineering report
- refactor a React component into hooks
- review a PR using team rules
- fix lint errors and add tests

10.1 Skill Structure

Typical layout:

```
.skills/  
├─ add-api-endpoint/  
│   ├── SKILL.md  
│   ├── templates/  
│   │   ├── handler.ts.tmpl  
│   │   └── test.ts.tmpl  
│   └── scripts/  
│       └── update-openapi.sh
```

SKILL.md is the core:

```
---  
name: add-api-endpoint  
description: Use when adding a new REST API endpoint, route, handler, service, tests, and  
---  
  
# Add API Endpoint  
  
## Use When  
  
The user asks to add a new HTTP endpoint.  
  
Do not use for:  
- modifying an existing endpoint  
- adding a gRPC method  
  
## Inputs  
  
- HTTP method  
- path  
- request schema  
- response schema  
  
## Steps  
  
1. Add the route.  
2. Add the handler.  
3. Add the service method.  
4. Add repository access if needed.
```

5. Add tests for success, validation failure, auth failure, and internal error.
6. Update OpenAPI.
7. Run the test command.

Project Rules

- Use the existing error wrapper.
- Auth is enforced in middleware.
- Times are RFC3339.

Done When

- [] tests pass
- [] OpenAPI is updated
- [] at least one manual request was verified

10.2 Skill Quality Rules

Good skills:

- have a clear trigger description
- use numbered steps
- reference exact local paths and commands
- capture project-specific conventions
- include acceptance criteria
- stay short enough to execute

Bad skills:

- are vague
- are 1,000-line essays
- overlap with other skills
- hard-code one-off business details
- reference old paths after the project changes

10.3 Skill vs AGENTS.md vs Spec

Artifact	Scope	Purpose
AGENTS.md	whole project	project map, rules, commands, red lines
skill	repeated task type	standard operating procedure
spec	one feature	concrete requirements and acceptance

Analogy:

- AGENTS.md is the employee handbook
- a skill is an SOP
- a spec is the ticket

11. System Prompts vs User Prompts

This distinction improves every agent interaction.

System prompt: durable behavior and rules for the agent.

User prompt: the current task.

11.1 What Belongs in the System Prompt

Put stable rules there:

- role and expertise
- behavior rules
- output format
- tool usage rules
- project knowledge
- security constraints
- recurring coding conventions

Example:

You are a senior Go backend engineer.

You are good at:

- PostgreSQL
- distributed systems
- table-driven tests

You must:

- wrap errors with context
- never use float for money
- run tests before finalizing
- ask before editing migrations

11.2 What Belongs in the User Prompt

Put current work there:

- the task
- the relevant data
- temporary constraints
- acceptance criteria

Example:

Context: We are implementing account deletion. The spec is specs/003-delete.md.

Task: Implement Phase 2, the background cleanup job.

Constraints:

- use cron, not a worker queue

- make the job idempotent
- do not change the Phase 1 API

Acceptance:

- tests pass
- one manual cleanup run marks expired requests completed

11.3 Decision Test

Ask:

Will the next session need this rule?

If yes, put it in AGENTS.md, a project instruction, or a skill.

If no, keep it in the user prompt.

11.4 Good User Prompt Formula

Context + Task + Constraints + Acceptance

Do not paste large files. Reference paths:

Read specs/003-data-export.md, especially the idempotency section.

State what not to do early:

Add deleteAccount.

Do not refactor UserService.

Do not change public method signatures.

Do not modify schema.

Keep each prompt to one main task.

11.5 Prompt Anti-Patterns

Anti-Pattern	Result	Fix
system prompt is a novel repeating project background in every prompt	rules get buried wasted context	short numbered rules move to system
putting one-time tasks in system	future sessions polluted	keep tasks in user prompts
no acceptance criteria	unclear done state	define verification
only saying what to do	agent overreaches	include boundaries

System prompt is the agent's gravity. User prompt is today's destination.

12. CI/CD for Agent-Written Code

Agents can produce code faster than you can review. CI is the machine reviewer that catches basic failures before they reach main.

CI should catch:

- tests
- lint
- type errors
- formatting
- build failures
- dependency vulnerabilities
- secret leaks
- coverage regressions

12.1 Minimal GitHub Actions CI

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: '20'

      - run: npm ci
      - run: npm run lint
      - run: npm test
      - run: npm run build
```

Adjust commands to your stack.

12.2 Add Local CI

Agents should run the same checks locally before pushing.

Example Makefile:

```
ci-local: lint test build
    @echo "local CI passed"
```

```
lint:
  npm run lint

test:
  npm test

build:
  npm run build
```

Then write in AGENTS.md:

CI Rules

Before committing, run *`make ci-local`*.
If CI fails, inspect the failing step and reproduce locally before changing code.
Do not guess fixes from partial logs.

12.3 AI in CI

AI can also run inside CI for:

- PR review
- changelog generation
- issue labeling
- documentation checks

Non-interactive CI agents need stricter rules:

- if ambiguous, fail and explain
- never push directly to main
- limit changed files
- write structured PR comments
- do not hide failures

12.4 CD Guardrails

Never let AI directly deploy to production.

Prefer:

```
feature branch -> CI
PR -> CI + AI review + human review
main -> staging deploy
staging smoke tests
human approval
production deploy
```

The human approval is not just code review. It is a final operational brake.

12.5 CI/CD Anti-Patterns

Anti-Pattern	Result	Fix
no CI	bad agent code reaches main	add minimal CI
local and CI commands differ	“works locally” failures	one local CI entrypoint
CI too slow	people ignore it	split fast and slow checks
AI deploys production	high operational risk	require human approval
secrets in workflow files	leaked credentials	use GitHub Secrets
agents modify CI silently	checks get bypassed	require explicit review

CI is not extra ceremony. For agent-written code, it is part of the control system.

13. Testing Code and Testing Agent Behavior

Vibe coding has two layers of testing.

A. Testing normal code

You verify functions, APIs, UI flows, and integrations.

B. Testing agent behavior

You verify prompts, skills, tool calls, and agent loops.

13.1 Testing Normal Code

Three useful modes:

You write tests, agent implements

Implement MergeIntervals.

Here are the cases:

- [] -> []
- [[1,3],[2,6],[8,10]] -> [[1,6],[8,10]]
- [[1,2],[3,4]] -> unchanged
- [[1,10],[2,3]] -> [[1,10]]

This gives you control over correctness.

Agent writes tests first, you review

Before implementing, write the test cases you intend to cover.

Wait for my approval before writing production code.

Reviewing tests is often easier than reviewing implementation.

Agent lists edge cases

Before writing tests, list edge cases and invalid inputs.

I will choose which ones matter.

The agent is good at enumeration. You still make the engineering judgment.

Do not let the agent implement first and then write tests that fit its implementation.

13.2 Testing Agent Behavior

When the product includes prompts, skills, or agent loops, function tests are not enough.

Agent behavior is:

- non-deterministic
- hard to define with a single correct answer
- affected by tool calls and hidden state
- vulnerable to prompt wording changes

Use case-driven testing.

13.3 Case-Driven Agent Testing

Create a case file:

```
# Case: Monthly spending question

## Input Prompt

"How much did I spend last month? Break it down by category."

## Context

- user_id=42
- database has 30 transactions across August and September
- current date is 2024-09-15

## Expected Behavior

1. Agent calls `get_transactions` with:
  - user_id=42
  - start=2024-08-01
  - end=2024-08-31
2. Agent does not call `get_user_profile`.
3. Response includes:
  - correct total
  - category breakdown
  - currency format
  - no more than 5 lines

## Failure Signals

- wrong date range
- extra tool calls
- incorrect total
- wrong currency
```

Run it 2-3 times. Save evidence:

```
runs/case01/
└─ case.md
```

```
|— run1-tui.txt
|— run1-tools.json
|— run1-db-diff.txt
|— run2-tools.json
|— run3-tools.json
|— observation.md
```

Write observations manually:

Stability

- tool call was correct in runs 1 and 2
- run 3 used start=2024-08-15, which is wrong
- run 2 made an unnecessary profile call

Main Problems

1. Date range is unstable.
2. Currency format differs across runs.

Then give the evidence to an agent:

This is a prompt debugging task.

Read case.md, observation.md, run*-tools.json, and the current system prompt.

The main issue is a 1/3 failure rate in date range selection.

Give 2-3 hypotheses before changing the prompt.

After changing the prompt, rerun the same cases.

13.4 Agent Behavior Testing Rules

Rule	Reason
run each case at least 2-3 times	one success can be luck
keep a written case file	memory is unreliable
capture tool calls, not only final text	hidden behavior matters
do human observation	agents justify their own output
rerun old cases after prompt changes	prevent regressions

Agent behavior is statistical. Your tests need to measure stability, not just one clean output.

14. Advanced Principles

14.1 Make the Agent Speak First

For important work, first ask for understanding and plan:

I want to add user search.

Before editing, tell me:

- how you would implement it

- files you expect to change
- edge cases
- decisions you need from me

The extra minute often saves a bad half-hour.

14.2 Commit Frequently

Use git as your undo system.

After each independent change, show the diff and commit it with a clear message.

Small commits make agent experiments safer.

14.3 Reject “Looks Correct”

Agent code can look professional and still be wrong. Names, comments, and structure do not prove behavior.

Tests and runtime verification do.

14.4 Put Knowledge in Files

When an agent discovers something useful, ask:

Will future me or a future agent need this?

If yes, store it in:

- AGENTS.md
- docs/
- specs/
- an ADR
- a focused code comment

14.5 Stop Early When Direction Is Wrong

Do not keep negotiating with a bad direction.

Stop. This is not the right approach.

Summarize what went wrong, reset the plan, and propose a smaller path.

Interrupting is a core skill.

15. A Complete Workflow Example

A realistic feature flow might look like this.

Day 1 Morning: Spec

We are building "user data export."
Read AGENTS.md and specs/template.md.
Draft specs/003-data-export.md.
Ask every unclear question before writing implementation details.

The agent asks questions. You answer. The agent writes the spec. You review and commit.

Day 1 Afternoon: Design

Start a new session because the exploration context is heavy:

Read specs/003-data-export.md.
Design the implementation.
Do not write code yet.
Tell me which tables, APIs, background jobs, and files are involved.

You iterate on the design, then store it:

Write the final design to docs/design/003-data-export.md.

Day 2: Phase 1 Implementation

Start fresh:

Read AGENTS.md, the spec, and the design.
Implement Phase 1: create export request API.
After each file, show the diff before continuing.

If a complex database question appears:

Use a subagent to research large PostgreSQL export patterns.
Return three options and tradeoffs. Do not edit files.

After implementation:

- run tests
- commit
- write notes

Write docs/notes/003-data-export-phase1.md with progress, changed files, validation, and notes.

Day 3: Phase 2

Read AGENTS.md, spec, design, and phase1 notes.
Continue with Phase 2.
Restate the plan before editing.

Each session has a narrow purpose. Files carry memory across sessions.

16. Anti-Patterns Checklist

Anti-Pattern	Result	Better Practice
no spec before coding	wrong direction	write spec first
one session all day	quality collapse	switch at milestones
stale AGENTS.md	repeated mistakes	update it weekly
coding immediately in an inherited repo	inconsistent changes	explore and run locally first
trusting generated AGENTS.md blindly	missing implicit rules	review and improve it
agent reviews its own code	weak review	use another role
large task without phases	unreviewable diff	split and commit
no tests	plausible broken code	test first or review tests first
trusting “tests passed” without output	false confidence	inspect command output
treating chat as memory	forgotten decisions	write files
multiple agents in one directory	file conflicts	use worktrees
worktree without setup	missing env/deps	write setup script
overloaded context	degraded agent	write handoff and restart
pasting huge files	context waste	reference paths
repeating the same instructions	inconsistent work	create a skill
skill written as prose	poor execution	numbered steps
tests after implementation	tests fit bugs	test before code
agent behavior tested once	unstable prompts	run cases 2-3 times
prompt debugging without evidence	guessing	save tool calls and observations
direction is wrong but session continues	messy rollback	stop and reset
project background in every user prompt	wasted context	move to system prompt
one-time task in system prompt	future pollution	keep it in user prompt
bad .gitignore	garbage or secrets committed	fix ignore rules first
committed secrets	history leak	rotate secrets and rewrite history
no CI	broken code reaches main	add minimal CI
local checks differ from CI	surprise failures	one local CI command
AI deploys production	operational risk	require human approval
using agents when workflow is enough	unpredictable cost	define workflow
no evaluator exit condition	endless loop	max iterations

Closing

Vibe Coding is not about abandoning engineering discipline. It moves your discipline up a level.

You become the person who writes the spec, sets the boundaries, gives the context, reviews the diff, designs the workflow, and decides when to stop.

The agent writes many of the characters. You still own the software.